

Gema/Kidrove Deployment Guide - Multi-Region

Complete production deployment guide for the Gema Event Management Platform with multi-region support across kidrove.com, kidrove.in, and kidrove.ae.

Previous deployment guide backed up to: [publish.md.old](#)

Overview

Architecture

- **3 Regional Domains:** kidrove.com (UAE), kidrove.in (India), kidrove.ae (UAE)
- **9 Total Domains:** 3 main domains + 3 www subdomains + 3 API subdomains
- **Single Backend:** One Node.js API server serving all regions on port 5000
- **Runtime API Detection:** Frontend automatically routes to correct API subdomain based on current domain
- **Server:** Hostinger KVM1 VPS (1 vCPU, 4GB RAM, 50GB NVMe) at 93.127.185.245

Technology Stack

- **Backend:** Node.js 20.x + Express.js + TypeScript + PM2
 - **Frontend:** React 18 + TypeScript + Vite
 - **Database:** MongoDB Atlas (cloud) + Redis (local caching)
 - **Web Server:** NGINX with HTTP/2 and SSL/TLS 1.3
 - **SSL:** Let's Encrypt via Certbot (6 certificates, auto-renewal)
 - **Payments:** Stripe with Connect
 - **Storage:** Cloudinary
-

Prerequisites

Before beginning deployment:

- [] **VPS Server:** Hostinger KVM1 or equivalent (Ubuntu 22.04, 4GB RAM minimum)
 - [] **Domain Names:** kidrove.com, kidrove.in, kidrove.ae registered and accessible
 - [] **GitHub Access:** Repository access to GEMA-Project
 - [] **MongoDB Atlas Account:** Free tier M0 or paid tier
 - [] **Stripe Account:** Test mode (switch to live later)
 - [] **Cloudinary Account:** For media storage
 - [] **Email Service:** Hostinger email or Gmail with app password
 - [] **SSH Client:** PowerShell/Git Bash/PuTTY for Windows
-

Phase 1: Server Initial Setup

Step 1.1: SSH to Your Server

From your local machine (Windows PowerShell, Git Bash, or CMD):

```
ssh root@93.127.185.245
```

Enter your server password when prompted.

Step 1.2: Update System Packages

```
# Update package lists and upgrade existing packages
sudo apt update && sudo apt upgrade -y

# Install essential build tools
sudo apt install -y curl wget git build-essential
```

Step 1.3: Install Node.js 20.x

```
# Download and run NodeSource setup script
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# Install Node.js (includes npm)
sudo apt install -y nodejs

# Verify installation
node -v # Should show v20.x.x
npm -v  # Should show 10.x.x or higher
```

Step 1.4: Install PM2 Process Manager

```
# Install PM2 globally
sudo npm install -g pm2

# Configure PM2 to start on system boot
pm2 startup systemd

# Copy and run the command it outputs (it will be specific to your system)
# Example: sudo env PATH=$PATH:/usr/bin /usr/lib/node_modules/pm2/bin/pm2 startup systemd -u root --hp /root

# After running that command, save PM2 configuration
pm2 save
```

Step 1.5: Install NGINX Web Server

```
# Install NGINX
sudo apt install -y nginx

# Start and enable NGINX
sudo systemctl start nginx
sudo systemctl enable nginx

# Verify NGINX is running
sudo systemctl status nginx
```

output

[illegible]

Step 1.7: Install Certbot for SSL

```
# Install Certbot and NGINX plugin
sudo apt install -y certbot python3-certbot-nginx
```

Step 1.8: Configure Firewall

```
# Allow SSH (port 22)
sudo ufw allow 22/tcp

# Allow HTTP (port 80)
sudo ufw allow 80/tcp

# Allow HTTPS (port 443)
sudo ufw allow 443/tcp

# Enable firewall
sudo ufw --force enable

# Check firewall status
sudo ufw status verbose
```

Output

```
root@srv1143065:~# sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
22/tcp (OpenSSH) ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
22/tcp (OpenSSH (v6)) ALLOW IN Anywhere (v6)
```

Expected output: Ports 22, 80, and 443 should show as ALLOW.

Phase 2: DNS Configuration

Step 2.1: Configure DNS Records

In your domain registrar's DNS management panel, create these A records:

For kidrove.com:

Type:	A	Name:	@	Value:	93.127.185.245
Type:	A	Name:	www	Value:	93.127.185.245
Type:	A	Name:	api	Value:	93.127.185.245

For kidrove.in:

Type:	A	Name:	@	Value:	93.127.185.245
Type:	A	Name:	www	Value:	93.127.185.245
Type:	A	Name:	api	Value:	93.127.185.245

For kidrove.ae:

Type:	A	Name:	@	Value:	93.127.185.245
Type:	A	Name:	www	Value:	93.127.185.245
Type:	A	Name:	api	Value:	93.127.185.245

Total: 9 A records

Step 2.2: Verify DNS Propagation

Wait 1-24 hours for DNS propagation, then verify:

```
# Check each domain
nslookup kidrove.com
nslookup www.kidrove.com
nslookup api.kidrove.com
nslookup kidrove.in
nslookup www.kidrove.in
nslookup api.kidrove.in
nslookup kidrove.ae
nslookup www.kidrove.ae
nslookup api.kidrove.ae
```

Each should return: Address: 93.127.185.245

Important: Do not proceed until DNS propagation is complete.

Phase 3: Repository Setup

Step 3.1: Create Project Directory

```
# Create directory
sudo mkdir -p /var/www/GEMA-Project

# Navigate to directory
cd /var/www/GEMA-Project
```

Step 3.2: Clone Repository

```
# Clone from GitHub (using backend_auth branch)
sudo git clone -b backend_auth https://github.com/EshaanManchanda/GEMA-Project.git .

# Set proper permissions
sudo chown -R $USER:$USER /var/www/GEMA-Project

# Verify files
ls -la
# Should show: backend/, frontend/, ecosystem.config.js, etc.
```

Phase 4: Backend Configuration

Step 4.1: Navigate to Backend Directory

```
cd /var/www/GEMA-Project/backend
```

Step 4.2: Create .env File

```
nano .env
```

Paste this configuration (update the placeholder values with your actual credentials):

```
#####
# GEMA Backend - Production Environment Configuration
#####

# =====
# SERVER CONFIGURATION
# =====
PORT=5000
NODE_ENV=production

# =====
# DATABASE CONFIGURATION
# =====
# MongoDB Atlas Connection String
MONGODB_URI=mongodb+srv://your_username:your_password@cluster0.xxxxx.mongodb.net/kidrove

# Connection Pool Settings (Optimized for KV1)
MONGODB_MAX_POOL_SIZE=50
```

```
MONGODB_MIN_POOL_SIZE=5
MONGODB_MAX_IDLE_TIME_MS=60000
MONGODB_CONNECT_TIMEOUT_MS=30000
MONGODB_SOCKET_TIMEOUT_MS=45000
MONGODB_SERVER_SELECTION_TIMEOUT_MS=30000

# =====
# JWT CONFIGURATION
# =====
# Generate secure secrets: openssl rand -base64 64
JWT_SECRET=your_jwt_secret_here
JWT_EXPIRES_IN=7d
JWT_REFRESH_SECRET=your_refresh_secret_here
JWT_REFRESH_EXPIRES_IN=30d

# Admin Registration Secret
ADMIN_SECRET_KEY=your_admin_secret_here

# =====
# FIREBASE ADMIN SDK
# =====
FIREBASE_PROJECT_ID=your-firebase-project-id
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\\nYOUR_KEY_HERE\\n-----END PRIVATE KEY-----\\n"
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxxxx@your-project.iam.gserviceaccount.com

# =====
# CORS CONFIGURATION - MULTI-REGION
# =====
FRONTEND_URL=https://kidrove.com
ADDITIONAL_ALLOWED_ORIGINS=https://www.kidrove.com,https://kidrove.in,https://www.kidrove.in,https://kidrove.ae,https://www.kidrove.ae

# =====
# RATE LIMITING
# =====
RATE_LIMIT_WINDOW_MS=15
RATE_LIMIT_MAX=100

# =====
# PAYMENT CONFIGURATION
# =====
# Stripe API Keys (use test keys initially)
STRIPE_SECRET_KEY=sk_test_your_stripe_secret_key
STRIPE_PUBLISHABLE_KEY=pk_test_your_stripe_publishable_key
STRIPE_WEBHOOK_SECRET=whsec_your_webhook_secret

# =====
# EMAIL CONFIGURATION
# =====
EMAIL_SERVICE=hostinger
EMAIL_HOST=smtp.hostinger.com
EMAIL_PORT=587
EMAIL_USERNAME=contact@kidrove.com
EMAIL_PASSWORD=your_email_password
EMAIL_FROM=contact@kidrove.com
EMAIL_FROM_NAME=Kidrove

# =====
# REDIS CONFIGURATION
# =====
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
REDIS_DB=0
REDIS_QUEUE_DB=0
REDIS_TLS=false
DISABLE_REDIS=false

# =====
```

```
# JOB PROCESSING
# =====
ENABLE_JOBS=true

# =====
# CLOUDINARY CONFIGURATION
# =====
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
CLOUDINARY_URL=cloudinary://your_api_key:your_api_secret@your_cloud_name

UPLOAD_PROVIDER=cloudinary
CLOUDINARY_UPLOAD_PRESET=ml_default
CLOUDINARY_SECURE=true
```

Save and exit: Ctrl+X, then Y, then Enter

Critical: Update all placeholder values with your actual credentials!

Step 4.3: Verify Configuration

```
# Check .env file exists
ls -la .env

# Verify critical variables (without showing values)
grep -q "MONGODB_URI" .env && echo "✓ MongoDB configured" || echo "✗ Missing MongoDB"
grep -q "JWT_SECRET" .env && echo "✓ JWT configured" || echo "✗ Missing JWT"
grep -q "STRIPE_SECRET_KEY" .env && echo "✓ Stripe configured" || echo "✗ Missing Stripe"
grep -q "FRONTEND_URL" .env && echo "✓ CORS configured" || echo "✗ Missing CORS"
```

You should see four checkmarks (✓).

Step 4.4: Install Dependencies and Build

```
# Install dependencies (takes 2-3 minutes)
npm install

# Build TypeScript to JavaScript
npm run build

# Verify build succeeded
ls -la dist/
# Should show: server.js and other compiled files
```

Phase 5: Frontend Configuration

Step 5.1: Navigate to Frontend Directory

```
cd /var/www/GEMA-Project/frontend
```

Step 5.2: Create .env.production File

```
nano .env.production
```

Paste this configuration:

```
# =====
# GEMA Frontend - Production Environment
# =====

# API Configuration
# Note: Runtime API detection is used in production (frontend/src/config/api.ts)
# The frontend automatically detects the domain and uses the corresponding API:
# - kidrove.com → api.kidrove.com
# - kidrove.in → api.kidrove.in
# - kidrove.ae → api.kidrove.ae
# This VITE_API_BASE_URL is used as fallback for localhost development only
VITE_API_BASE_URL=https://api.kidrove.com/api
VITE_API_TIMEOUT=30000
VITE_BUILD_TARGET=production
NODE_ENV=production

# App Configuration
VITE_APP_NAME=Kidrove
VITE_APP_VERSION=1.0.0
VITE_APP_DESCRIPTION=Event Management and Booking Platform
VITE_APP_URL=https://kidrove.com

# Payment Configuration (use test keys initially)
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_your_stripe_publishable_key

# Test Mode Configuration
VITE_ENABLE_TEST_PAYMENTS=false
VITE_PREFERRED_PAYMENT_METHOD=stripe
VITE_PAYMENT_REGION=AE
VITE_PAYMENT_ENVIRONMENT=production

# File Upload Configuration
VITE_MAX_FILE_SIZE=10485760
VITE_ALLOWED_FILE_TYPES=image/jpeg,image/png,image/webp,image/gif
VITE_CLOUDINARY_CLOUD_NAME=your_cloud_name
VITE_CLOUDINARY_UPLOAD_PRESET=ml_default
VITE_CLOUDINARY_API_KEY=your_api_key

# Firebase Configuration
VITE_FIREBASE_API_KEY=your_firebase_api_key
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseio.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.firebaseiostorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
VITE_FIREBASE_APP_ID=your_app_id
VITE_FIREBASE_MEASUREMENT_ID=your_measurement_id

# Production Settings
VITE_ENABLE_DEVTOOLS=false
VITE_LOG_LEVEL=error
VITE_ENABLE MOCK_API=false

# Feature Flags
VITE_ENABLE_CHAT=true
VITE_ENABLE_NOTIFICATIONS=true
VITE_ENABLE_REVIEWS=true
VITE_ENABLE_FAVORITES=true
VITE_ENABLE_DARK_MODE=true

# Localization
VITE_DEFAULT_LANGUAGE=en
VITE_DEFAULT_CURRENCY=AED
VITE_DEFAULT_TIMEZONE=Asia/Dubai

# Security
VITE_ENABLE_CSP=true
VITE_ENABLE_HTTPS_REDIRECT=true
VITE_SECURE_COOKIES=true
```


Save and exit: Ctrl+X, then Y, then Enter

Important: Update placeholder values with your actual credentials.

Step 5.3: Install Dependencies and Build

```
# Install dependencies (takes 3-5 minutes)
npm install

# Build for production
npm run build

# Verify build succeeded
ls -la dist/
# Should show: index.html, assets/, etc.

# Check build size
du -sh dist/
```

Phase 6: NGINX Configuration

Step 6.1: Create NGINX Snippet Files

These reusable configuration snippets will be included in the main config:

Create SSL parameters snippet:

```
sudo nano /etc/nginx/snippets/ssl-params.conf
```

Paste:

```
# SSL/TLS Configuration Parameters
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;
ssl_prefer_server_ciphers on;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 10m;
ssl_stapling on;
ssl_stapling_verify on;
resolver 8.8.8.8 8.8.4.4 valid=300s;
resolver_timeout 5s;
```

Save and exit.

Create security headers snippet:

```
sudo nano /etc/nginx/snippets/security-headers.conf
```

Paste:

```
# Security Headers Configuration
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

Save and exit.

Create API proxy snippet:

```
sudo nano /etc/nginx/snippets/api-proxy.conf
```

Paste:

```
# API Proxy Configuration
proxy_pass http://backend_api;
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection 'upgrade';
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
proxy_connect_timeout 60s;
proxy_send_timeout 60s;
proxy_read_timeout 60s;
proxy_cache_bypass $http_upgrade;
proxy_buffering off;
proxy_request_buffering off;
```

Save and exit.

Step 6.2: Upload nginx-multi-region.conf

From your local machine, upload the nginx configuration:

```
scp E:\coding\gema\nginx-multi-region.conf root@93.127.185.245:/etc/nginx/sites-available/
```

Alternative (if using Git on server):

```
cd /var/www/GEMA-Project
git pull
sudo cp nginx-multi-region.conf /etc/nginx/sites-available/
```

Step 6.3: Enable NGINX Configuration

```
# Remove default site
sudo rm -f /etc/nginx/sites-enabled/default

# Enable new multi-region config
sudo ln -s /etc/nginx/sites-available/nginx-multi-region.conf /etc/nginx/sites-enabled/

# Test NGINX configuration
sudo nginx -t
```

Expected output:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

If test fails: Check error message for syntax errors or missing SSL certificate paths (we'll add certificates in next phase).

Phase 7: SSL Certificates

Step 7.1: Request SSL Certificates

Run these commands **in sequence** (one at a time):

```
# 1. kidrove.com + www.kidrove.com (SAN certificate)
sudo certbot certonly --nginx -d kidrove.com -d www.kidrove.com

# 2. api.kidrove.com
sudo certbot certonly --nginx -d api.kidrove.com

# 3. kidrove.in + www.kidrove.in (SAN certificate)
sudo certbot certonly --nginx -d kidrove.in -d www.kidrove.in

# 4. api.kidrove.in
sudo certbot certonly --nginx -d api.kidrove.in

# 5. kidrove.ae + www.kidrove.ae (SAN certificate)
sudo certbot certonly --nginx -d kidrove.ae -d www.kidrove.ae

# 6. api.kidrove.ae
sudo certbot certonly --nginx -d api.kidrove.ae
```

For each certificate, Certbot will ask

1. **Email address:** Enter your email for renewal notifications
2. **Terms of Service:** Type `Y` and press Enter
3. **Share email with EFF:** Type `Y` or `N` (your choice)

Expected output for each:

```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/[domain]/fullchain.pem
Key is saved at: /etc/letsencrypt/live/[domain]/privkey.pem
This certificate expires on [date 90 days from now].
```

Step 7.2: Verify All Certificates

```
sudo certbot certificates
```

Expected output: 6 certificates listed, all valid for 90 days.

Step 7.3: Test Auto-Renewal

```
sudo certbot renew --dry-run
```

Expected output:

```
Congratulations, all simulated renewals succeeded
```

Auto-renewal: Certbot creates a systemd timer that runs twice daily to renew certificates.

Step 7.4: Reload NGINX with SSL

```
# Test configuration again (now with SSL certificates)
sudo nginx -t

# Reload NGINX
sudo systemctl reload nginx

# Check NGINX status
sudo systemctl status nginx
```

Phase 8: PM2 Process Management

Step 8.1: Create Log Directories

```
sudo mkdir -p /var/log/GEMA-Project/backend
sudo chown -R $USER:$USER /var/log/GEMA-Project
```

Step 8.2: Start Backend with PM2

```
cd /var/www/GEMA-Project/backend

# Start using ecosystem config (starts both backend and worker)
pm2 start ecosystem.config.js --env production

# Save PM2 configuration
pm2 save

# Check status
pm2 status
```

Expected output:

id	name	mode	pid	uptime	🔄	status	cpu
0	gema-backend	fork	12345	5s	0	online	0%
1	gema-worker	fork	12346	5s	0	online	0%

Both processes should show **status: online**.

Step 8.3: View Logs

```
# View combined logs
pm2 logs

# View backend logs only
pm2 logs gemma-backend

# View last 50 lines
pm2 logs gemma-backend --lines 50

# View errors only
pm2 logs gemma-backend --err
```

Look for: "MongoDB connected successfully" and no errors.

Step 8.4: Configure PM2 Auto-Start

PM2 startup was already configured in Phase 1, but verify:

```
# Check PM2 startup
pm2 startup

# If needed, copy and run the command it outputs

# Save current processes to restore on reboot
pm2 save
```

Step 8.5: Setup PM2 Log Rotation

```
# Install PM2 log rotation module
pm2 install pm2-logrotate

# Configure rotation settings
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 10
pm2 set pm2-logrotate:compress true
pm2 set pm2-logrotate:dateFormat YYYY-MM-DD_HH-mm-ss
```

Phase 9: Testing & Verification

Step 9.1: Test Backend Locally

```
# Test health endpoint locally
curl http://localhost:5000/health

# Expected: {"status":"ok",...}
```

Step 9.2: Test All 9 Domains

Run these tests from your local machine or server:

Frontend domains (should return HTML):

```
curl -I https://kidrove.com
curl -I https://www.kidrove.com
curl -I https://kidrove.in
curl -I https://www.kidrove.in
curl -I https://kidrove.ae
curl -I https://www.kidrove.ae
```

API domains (should return JSON):

```
curl https://api.kidrove.com/api/health
curl https://api.kidrove.in/api/health
curl https://api.kidrove.ae/api/health
```

Expected Results:

Domain	Expected Response
https://kidrove.com	HTTP 200, HTML content
https://www.kidrove.com	HTTP 200, HTML content
https://kidrove.in	HTTP 200, HTML content
https://www.kidrove.in	HTTP 200, HTML content
https://kidrove.ae	HTTP 200, HTML content
https://www.kidrove.ae	HTTP 200, HTML content
https://api.kidrove.com/api/health	{ "status": "ok" }
https://api.kidrove.in/api/health	{ "status": "ok" }
https://api.kidrove.ae/api/health	{ "status": "ok" }

Step 9.3: Browser Testing

Open your browser and test:

1. **Frontend:** Navigate to <https://kidrove.com>
 - Should load without SSL warnings
 - Open Developer Tools (F12) → Console tab
 - Should see no CORS errors
 - Check Network tab for API calls
2. **Try other domains:** <https://kidrove.in> and <https://kidrove.ae>
 - Both should load identically to .com
3. **API Health:** Navigate to <https://api.kidrove.com/api/health>
 - Should display JSON: { "status": "ok", ... }

Step 9.4: CORS Testing

From browser console on each domain, test cross-origin requests:

```
// Test from kidrove.in
fetch('https://api.kidrove.in/api/health')
  .then(r => r.json())
  .then(console.log);
// Should NOT show CORS errors

// Test from kidrove.ae
fetch('https://api.kidrove.ae/api/health')
  .then(r => r.json())
  .then(console.log);
// Should NOT show CORS errors
```

Step 9.5: SSL Verification

```
# Check SSL certificate details for each domain
echo | openssl s_client -connect kidrove.in:443 -servername kidrove.in 2>/dev/null | openssl x509 -noout -dates
echo | openssl s_client -connect api.kidrove.in:443 -servername api.kidrove.in 2>/dev/null | openssl x509 -noout -dates
echo | openssl s_client -connect kidrove.ae:443 -servername kidrove.ae 2>/dev/null | openssl x509 -noout -dates
echo | openssl s_client -connect api.kidrove.ae:443 -servername api.kidrove.ae 2>/dev/null | openssl x509 -noout -dates
```

Step 9.6: Functional Testing Checklist

Test these features on each domain:

- ☐ User registration works
- ☐ User login works
- ☐ Events list displays
- ☐ Event details page loads
- ☐ Images load properly (Cloudinary)
- ☐ No console errors in browser
- ☐ API calls succeed (check Network tab)

Phase 10: Security Hardening

Step 10.1: Setup Fail2Ban

```
# Install Fail2Ban
sudo apt install -y fail2ban

# Create local configuration
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local

# Edit configuration
sudo nano /etc/fail2ban/jail.local
```

Find the [sshd] section and update:

```
[sshd]
enabled = true
port = 22
maxretry = 3
bantime = 3600
findtime = 600
```

Save and exit.

```
# Start and enable Fail2Ban
sudo systemctl start fail2ban
sudo systemctl enable fail2ban

# Check status
sudo fail2ban-client status sshd
```

Step 10.2: Configure Automated Backups

```
# Create backup script
sudo nano /root/backup-gema.sh
```

Paste:

```
#!/bin/bash
BACKUP_DIR="/root/backups"
DATE=$(date +%Y%m%d-%H%M%S)
mkdir -p $BACKUP_DIR

# Backup environment files
tar -czf $BACKUP_DIR/gema-env-$DATE.tar.gz \
  /var/www/GEMA-Project/backend/.env \
  /var/www/GEMA-Project/frontend/.env.production

# Backup PM2 ecosystem
cp /var/www/GEMA-Project/backend/ecosystem.config.js $BACKUP_DIR/ecosystem-$DATE.js

# Backup nginx config
cp /etc/nginx/sites-available/nginx-multi-region.conf $BACKUP_DIR/nginx-$DATE.conf

# Keep only last 7 backups
cd $BACKUP_DIR && ls -t gem-a-env-*.tar.gz | tail -n +8 | xargs -r rm

echo "Backup completed: $DATE"
```

Save and exit.

```
# Make executable
chmod +x /root/backup-gema.sh

# Test the backup
/root/backup-gema.sh

# List backups
ls -lh /root/backups/

# Add to cron (daily at 2 AM)
crontab -e
```

Add this line:

```
0 2 * * * /root/backup-gema.sh >> /var/log/gema-backup.log 2>&1
```

Save and exit.

Step 10.3: Secure MongoDB Atlas

1. Go to MongoDB Atlas dashboard
2. Navigate to **Network Access**
3. Remove 0.0.0.0/0 entry (if you added it for testing)
4. Keep only your server IP: 93.127.185.245/32

Step 10.4: Setup Auto-Updates

```
# Update all packages
sudo apt update && sudo apt upgrade -y

# Install unattended upgrades
sudo apt install -y unattended-upgrades

# Configure automatic security updates
sudo dpkg-reconfigure -plow unattended-upgrades
# Select "Yes" when prompted
```

Phase 11: Production Configuration

Step 11.1: Configure Stripe Webhooks

1. Login to Stripe Dashboard: <https://dashboard.stripe.com/webhooks>
2. Click "Add endpoint"
3. **Endpoint URL:** <https://api.kidrove.com/api/webhooks/stripe>
4. **Description:** Kidrove Production Webhook
5. **Events to send:** Select:

- payment_intent.succeeded
- payment_intent.payment_failed
- checkout.session.completed
- checkout.session.expired
- customer.subscription.created
- customer.subscription.updated
- customer.subscription.deleted

6. Click "Add endpoint"
7. Click on the webhook and reveal the **Signing secret** (starts with whsec_)
8. Copy the signing secret

Update backend .env:

```
cd /var/www/GEMA-Project/backend
nano .env
```

Find STRIPE_WEBHOOK_SECRET= and add your secret:

```
STRIPE_WEBHOOK_SECRET=whsec_your_actual_secret_here
```

Save and exit.

```
# Restart backend
pm2 restart gema-backend

# Check logs
pm2 logs gema-backend --lines 20
```

Step 11.2: Verify Environment Variables

```
cd /var/www/GEMA-Project/backend

# Check all critical variables are set
grep MONGODB_URI .env
grep JWT_SECRET .env
grep STRIPE_WEBHOOK_SECRET .env
grep FRONTEND_URL .env
grep ADDITIONAL_ALLOWED_ORIGINS .env
```

Ensure all show values (not empty).

Step 11.3: Optional - Setup PM2 Plus Monitoring

```
# Sign up at https://app.pm2.io
# Get your keys and run:
pm2 link <secret_key> <public_key>

# Monitor at: https://app.pm2.io
```

Phase 12: Maintenance & Updates

Deployment Update Workflow

When you push new code to GitHub:


```
# SSH to server
ssh root@93.127.185.245

# Navigate to project
cd /var/www/GEMA-Project

# Pull latest code
git pull origin backend_auth

# Update backend
cd backend
npm install
npm run build
pm2 reload gema-backend

# Update frontend
cd ../frontend
npm install
npm run build
sudo systemctl reload nginx

# Verify
pm2 status
pm2 logs gema-backend --lines 20
```

PM2 Commands Reference

```
# View status
pm2 status

# View logs
pm2 logs # All processes
pm2 logs gema-backend # Backend only
pm2 logs gema-backend --lines 100 # Last 100 lines
pm2 logs gema-backend --err # Errors only

# Restart services
pm2 restart gema-backend # With downtime
pm2 restart gema-worker
pm2 restart all

# Reload (zero-downtime)
pm2 reload gema-backend # Recommended for updates
pm2 reload all

# Stop services
pm2 stop gema-backend
pm2 stop all

# Monitor resources
pm2 monit

# Clear logs
pm2 flush

# View detailed info
pm2 info gema-backend
```

NGINX Commands Reference

```
# Test configuration
sudo nginx -t

# Reload configuration (zero-downtime)
sudo systemctl reload nginx

# Restart NGINX
sudo systemctl restart nginx

# Check status
sudo systemctl status nginx

# View logs
sudo tail -f /var/log/nginx/error.log
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/kidrove-com-access.log
sudo tail -f /var/log/nginx/api-kidrove-com-error.log
```

SSL Certificate Management

```
# Check certificates
sudo certbot certificates

# Renew certificates manually
sudo certbot renew

# Force renewal (if needed)
sudo certbot renew --force-renewal

# Test auto-renewal (dry-run)
sudo certbot renew --dry-run

# Check renewal timer status
sudo systemctl status certbot.timer
```

Monitoring Commands

```
# Check disk space
df -h

# Check memory usage
free -h

# Check system resources
htop # Install with: sudo apt install htop

# Check running processes
ps aux | grep node
ps aux | grep nginx

# Check ports
sudo netstat -tuln | grep -E ':(80|443|5000|6379)'
# or
sudo ss -tuln | grep -E ':(80|443|5000|6379)'

# Check firewall
sudo ufw status verbose

# View system logs
sudo journalctl -u nginx -n 50
sudo journalctl -u pm2-root -n 50
sudo journalctl -xe
```

Database Management

```
# Check Redis
redis-cli ping # Should return "PONG"
redis-cli info # Detailed Redis info

# Connect to MongoDB Atlas
# Use MongoDB Compass with your connection string
# Or mongosh (if installed):
mongosh "mongodb+srv://your-connection-string"
```

Backup & Restore

```
# Manual backup
/root/backup-gema.sh

# List backups
ls -lh /root/backups/

# Restore backup
cd /root/backups
tar -xzf gema-env-YYYYMMDD-HHMMSS.tar.gz -C /
pm2 restart all
```

Troubleshooting Guide

Backend Not Starting

Symptoms: PM2 shows process as errored or constantly restarting

Solutions:

```
# Check logs
pm2 logs gema-backend --lines 100

# Common issues:

# 1. MongoDB connection error
# Check .env MONGODB_URI is correct
grep MONGODB_URI /var/www/GEMA-Project/backend/.env

# 2. Port already in use
sudo lsof -i :5000
# If something is using port 5000, stop it

# 3. Missing dependencies
cd /var/www/GEMA-Project/backend
npm install

# 4. Build errors
npm run build
# Check output for TypeScript errors

# 5. Redis connection error
redis-cli ping
# If fails, restart Redis:
sudo systemctl restart redis-server
```

Frontend Not Loading

Symptoms: Browser shows 502 Bad Gateway or blank page

Solutions:

```
# Check NGINX logs
sudo tail -50 /var/log/nginx/error.log
sudo tail -50 /var/log/nginx/kidrove-com-error.log

# Verify dist folder exists
ls -la /var/www/GEMA-Project/frontend/dist/
# Should contain: index.html, assets/, etc.

# Rebuild if needed
cd /var/www/GEMA-Project/frontend
npm run build
sudo systemctl reload nginx

# Check NGINX syntax
sudo nginx -t
```

CORS Errors

Symptoms: Browser console shows "CORS policy blocked"

Solutions:

```
# Check backend CORS settings
cd /var/www/GEMA-Project/backend
grep FRONTEND_URL .env
grep ADDITIONAL_ALLOWED_ORIGINS .env

# Should show:
# FRONTEND_URL=https://kidrove.com
#
ADDITIONAL_ALLOWED_ORIGINS=https://www.kidrove.com,https://kidrove.in,https://www.kidrove.in,https://kidrove.ae,https://www.kidrove.ae

# Update if needed and restart
pm2 restart gema-backend

# Check backend logs for CORS debug messages
pm2 logs gema-backend | grep -i cors
```

SSL Issues

Symptoms: "Your connection is not private" warning in browser

Solutions:

```
# Check certificate status
sudo certbot certificates

# All 6 certificates should be valid

# If expired, renew
sudo certbot renew

# If renewal fails, check DNS is correct
nslookup kidrove.in
nslookup api.kidrove.in
nslookup kidrove.ae
nslookup api.kidrove.ae

# Test nginx config
sudo nginx -t

# Reload nginx
sudo systemctl reload nginx
```

High Memory Usage

Symptoms: PM2 shows high memory usage, server becomes slow

Solutions:

```
# Check PM2 memory
pm2 status

# Restart processes to free memory
pm2 restart all

# Check system memory
free -h

# If consistently high, consider:
# 1. Adjusting max_memory_restart in ecosystem.config.js
# 2. Upgrading server RAM
# 3. Optimizing database queries
```

Database Connection Errors

Symptoms: Backend logs show MongoDB connection errors

Solutions:

```
# Check MongoDB Atlas:
# 1. Cluster is running (not paused)
# 2. IP whitelist includes server IP: 93.127.185.245
# 3. Connection string is correct

# Test connection from server
cd /var/www/GEMA-Project/backend
node -e "const mongoose = require('mongoose'); require('dotenv').config();
mongoose.connect(process.env.MONGODB_URI).then(() => console.log('Connected')).catch(err =>
console.error(err));"
```

Performance Optimization Tips

1. Enable Cloudflare CDN (Free)

1. Sign up at <https://cloudflare.com>
2. Add your domains (kidrove.com, kidrove.in, kidrove.ae)
3. Update nameservers at domain registrar
4. Benefits:
 - Global CDN caching
 - DDoS protection
 - Additional SSL features
 - Free tier sufficient for most needs

2. Monitor Performance

```
# Install monitoring tools
sudo apt install -y sysstat

# CPU usage
mpstat 1 5

# Disk I/O
iostat 1 5

# Memory usage
vmstat 1 5
```

3. Database Indexing

1. Review MongoDB Atlas Performance Advisor
2. Add indexes for frequently queried fields
3. Monitor slow queries

4. Redis Caching

Ensure Redis is properly utilized:

```
# Check Redis memory usage
redis-cli info memory

# Check cache hit rate
redis-cli info stats | grep keypace
```

Scaling Considerations

When Traffic Grows

Current Setup: 1 vCPU, 4GB RAM, handles ~100-500 concurrent users

Scaling Options:

1. **Vertical Scaling** (Upgrade server):
 - 2 vCPU, 8GB RAM → 500-2000 users
 - 4 vCPU, 16GB RAM → 2000-5000 users
2. **MongoDB Upgrade:**
 - M0 (free) → M10 (paid, \$0.08/hr)
 - Better performance, automated backups
3. **PM2 Cluster Mode** (when upgrading CPU):

```
# Edit ecosystem.config.js
instances: 'max' # Uses all CPU cores
exec_mode: 'cluster'
```

4. **Load Balancer** (multiple servers):
 - Use NGINX as load balancer
 - Or cloud load balancer (AWS ALB, etc.)
5. **Separate Worker Server:**
 - Move gema-worker to dedicated server
 - Reduces load on main API server

Appendix

Full Environment Variable Reference

See `backend/.env.production` for complete list of environment variables.

Critical variables:

- `MONGODB_URI`: MongoDB connection string
- `JWT_SECRET`: JWT signing secret
- `FRONTEND_URL`: Primary frontend domain
- `ADDITIONAL_ALLOWED_ORIGINS`: Additional CORS domains
- `STRIPE_SECRET_KEY`: Stripe API key
- `STRIPE_WEBHOOK_SECRET`: Stripe webhook secret
- `CLOUDINARY_*`: Cloudinary credentials

Port Reference

- **80**: HTTP (redirects to HTTPS)
- **443**: HTTPS (NGINX)
- **5000**: Backend API (internal, proxied by NGINX)
- **6379**: Redis (internal, localhost only)

DNS Records Summary

Required A Records (9 total):

For `kidrove.com`:

- `@` → 93.127.185.245

- `www` → 93.127.185.245
- `api` → 93.127.185.245

For `kidrove.in`:

- `@` → 93.127.185.245
- `www` → 93.127.185.245
- `api` → 93.127.185.245

For `kidrove.ae`:

- `@` → 93.127.185.245
- `www` → 93.127.185.245
- `api` → 93.127.185.245

SSL Certificates Summary

Required Certificates (6 total):

1. `kidrove.com` (+ `www.kidrove.com` as SAN)
2. `api.kidrove.com`
3. `kidrove.in` (+ `www.kidrove.in` as SAN)
4. `api.kidrove.in`
5. `kidrove.ae` (+ `www.kidrove.ae` as SAN)
6. `api.kidrove.ae`

Renewal: Automatic via systemd timer (twice daily check, renews 30 days before expiry)

Live URLs

Production URLs (all should work identically):

Frontend:

- `https://kidrove.com`
- `https://www.kidrove.com`
- `https://kidrove.in`
- `https://www.kidrove.in`
- `https://kidrove.ae`
- `https://www.kidrove.ae`

API:

- `https://api.kidrove.com/api`
- `https://api.kidrove.in/api`
- `https://api.kidrove.ae/api`

Health Checks:

- `https://api.kidrove.com/api/health`
- `https://api.kidrove.in/api/health`
- `https://api.kidrove.ae/api/health`

Support Resources

- **MongoDB Atlas:** `https://cloud.mongodb.com`
- **Stripe Dashboard:** `https://dashboard.stripe.com`
- **PM2 Documentation:** `https://pm2.keymetrics.io/docs`
- **Let's Encrypt:** `https://letsencrypt.org`
- **NGINX Documentation:** `https://nginx.org/en/docs`
- **Cloudinary:** `https://cloudinary.com/console`

Conclusion

You have successfully deployed the Gema/Kidrove platform with multi-region support!

Deployment Summary:

- ☒ 3 regional domains (`kidrove.com`, `kidrove.in`, `kidrove.ae`)
- ☒ 9 total domains configured (3 main + 3 `www` + 3 API)
- ☒ 6 SSL certificates (auto-renewal enabled)

- ☒ Single backend serving all regions
- ☒ Runtime API detection in frontend
- ☒ PM2 process management with auto-start
- ☒ NGINX reverse proxy with HTTP/2
- ☒ MongoDB Atlas + Redis caching
- ☒ Automated backups (daily at 2 AM)
- ☒ Security hardening (Fail2Ban, SSL, firewall)
- ☒ Stripe webhooks configured

Next Steps:

1. Test all features thoroughly on all 3 domains
2. Switch Stripe from test mode to live mode (when ready)
3. Setup monitoring (PM2 Plus or UptimeRobot)
4. Configure Google Analytics (optional)
5. Setup CDN (Cloudflare recommended)
6. Monitor performance and scale as needed

Regular Maintenance:

- Check PM2 logs daily: `pm2 logs gema-backend`
- Monitor SSL expiry: `sudo certbot certificates`
- Review backups weekly: `ls -lh /root/backups/`
- Update dependencies monthly: `npm update`
- Apply security updates: Automatic via `unattended-upgrades`

For issues or questions, refer to the **Troubleshooting Guide** section above.

Happy deploying! 🚀